1 캐릭터 기반 인터랙티브 전시형 콘텐츠 프로토타입 기획서

목표: Bluegarage 형 실시간 캐릭터 인터랙션과 덱스터류 몰입형 전시/실감형 콘텐츠 포트폴리오를 동시에 커버하는 저비용 MVP

현재 기준 장비	PC 1 대 + 휴대폰 1 대 (휴대폰은 입력/운영 패널 역할)
핵심 출력물	Unreal 실행 화면 + 휴대폰 웹 패널 + 1~2 분 데모 영상 + 구조 문서
MVP 범위	1 캐릭터 / 감정 상태 기반 반응 / 공간 연출 연동 / 실시간 명령 처리
확장 방향	2 캐릭터 상호작용, WebRTC·gRPC PoC, 음성 입력, 운영 로그 고도화

1. 프로젝트 정의

1-1. 프로젝트 한 줄 정의

관람객 또는 운영자가 휴대폰 웹 패널에서 입력한 명령에 따라, PC 모니터의 Unreal 캐릭터와 공간 연출이 실시간으로 반응하는 저비용 전시형 인터랙티브 콘텐츠 프로토타입을 제작한다.

1-2. 왜 이 방향이 최적해인가

초기 포트폴리오에서는 고가의 HMD, 모캡, LED 월, 전용 센서 없이도 "실시간 인터랙션 구조 설계 능력"과 "운영 가능한 콘텐츠 시스템 사고"를 동시에 보여주는 것이 가장 중요하다. 따라서 출력은 PC 모니터의 Unreal 화면이 맡고, 휴대폰은 입력 장치이자 운영 패널로 사용한다.

1-3. 타깃 포지셔닝

Bluegarage 관점에서는 실시간 캐릭터 인터랙션 시스템, 덱스터류 관점에서는 몰입형 전시·실감형 콘텐츠 프로토타입으로 읽히도록 설계한다.

2. 핵심 콘셉트

2-1. 사용자 경험 시나리오

관람객이 휴대폰으로 인사/감정/분위기/질문 버튼을 누르거나 텍스트를 입력하면, 캐릭터가 표정·애니메이션·대사·시선·카메라·조명·사운드로 반응한다. 화면은 단순 캐릭터 뷰가 아니라 "전시 씬"으로 구성해 공간 연출까지 같이 변하도록 만든다.

2-2. MVP 데모 흐름

① 대기 상태 → ② 휴대폰 접속 → ③ 인사 명령 → ④ 캐릭터 반응 → ⑤ 감정 또는 분위기 변경 → ⑥ 배경 연출 변화 → ⑦ 짧은 인터랙션 종료. 데모 길이는 1~2 분 수준으로 설계한다.

2-3. 반드시 지켜야 할 제한

MVP 에서는 1 캐릭터만 구현한다. 음성 스트리밍, 고급 립싱크 자동화, 얼굴 추적, 2 캐릭터 동기화, WebRTC·gRPC 실전 적용은 초기 필수 범위에서 제외한다. 단, 구조는 나중에 해당 기능을 얹을 수 있게 설계한다.

3. 시스템 구조 원칙

3-1. 1 캐릭터 MVP 이지만 2 캐릭터 확장 전제

현재 구현은 1 캐릭터 기준으로 진행하되, 내부 명령 구조와 상태 구조는 단일 캐릭터 전용으로 고정하지 않는다. 캐릭터 식별자(CharacterId), 반응 대상(TargetId), 연출 명령(StageCommand)을 분리하여 향후 2 캐릭터 또는 다중 오브젝트로 확장할 수 있게 한다.

3-2. 데이터 드리븐 설계

감정, 애니메이션, 대사, 카메라, 조명, 사운드 매핑을 코드에 하드코딩하지 말고 Data Table 또는 별도 ReactionData 구조로 관리한다. 명령이 들어오면 시스템이 해당 데이터를 조회해 실행하는 구조로 만든다.

3-3. 실시간 명령과 렌더링 책임 분리

입력 수신 계층(웹 패널/API/WebSocket), 도메인 계층(명령 해석/우선순위/쿨다운), Unreal 실행 계층(캐릭터/카메라/조명/사운드)으로 역할을 분리한다. 이렇게 해야 나중에 WebRTC·gRPC PoC 나 2 캐릭터 동기화를 붙일 때 구조를 유지할 수 있다.

4. 기능별 구성과 사용 기술

초기 포트폴리오에서는 “보여줄 수 있는 기능”과 “그 기능을 구현하는 최소 기술”을 짝으로 설계한다. 아래 표의 항목만 완성해도 충분히 설명 가능한 MVP 가 된다.

기능 영역	MVP 기능	권장 기술/구현 방식	확장 포인트
입력/제어	휴대폰 웹 패널에서 버튼·텍스트 입력	간단한 웹 UI + REST API 또는 WebSocket 이벤트	음성 입력, 관리자/관람객 모드 분리
캐릭터 반응	감정별 애니메이션·표정·대사 반응	Unreal Character Blueprint, Animation Blueprint, Data Table	2 캐릭터 동시 반응, 우선순위 큐
공간 연출	조명·카메라·사운드가 함께 변화	Level Sequence 또는 Blueprint 제어, Light/Camera/Sound Cue	씬 전환, 장면 프리셋, 이벤트 연쇄
상태 관리	입력 명령 해석, 쿨다운, 중복 방지	Command DTO, Reaction Resolver, State Manager	행동 큐, 인터럽트 정책, 동기화

실시간 통신	화면 반응 지연 최소화	초기: HTTP 또는 WebSocket / 추후: WebRTC·gRPC PoC	저지연 스트리밍, 다중 클라이언트
운영성	테스트 버튼, 로그 확인, 시연 모드	간단한 운영 패널, 로그 출력, 디버그 HUD	이벤트 로그 저장, 리플레이, 장애 대응
문서화	구조 설명과 시연 포장	아키텍처 다이어그램, 상태 전이도, 데모 영상	포트폴리오 사이트, 기술 회고

5. 상세 개발 우선순위와 구현 순서

기간은 고정하지 않고, 반드시 아래 순서대로 진행한다. 선행 단계가 안정화되지 않은 상태에서 다음 단계로 넘어가지 않는다.

5-1. 콘셉트 고정

- 전시 씬의 분위기, 캐릭터 1 종, 감정 상태 수(권장 4 개), 배경 모드 수(권장 2~3 개), 입력 종류(버튼/텍스트)를 확정한다.
- 데모 목표를 “관람객 입력 → 캐릭터 반응 → 공간 연출 변화”로 고정하고, 이 외 기능은 모두 후순위로 둔다.
- 2 캐릭터 확장 전제를 위해 Entity 개념을 먼저 도입한다. 현재는 Character_01 만 쓰더라도 자료구조는 CharacterId 를 받도록 정의한다.

5-2. Unreal 내부 반응 시스템만 먼저 구현

- 휴대폰 연결 전에 키보드 입력만으로 감정 변경이 동작하도록 만든다.
- Happy / Sad / Angry / Surprise 같은 상태를 입력하면 애니메이션, 표정, 대사 UI 가 바뀌게 한다.
- 이 단계의 완료 기준은 “입력이 들어오면 캐릭터가 즉시 다르게 보인다”이다.

5-3. 반응 데이터를 구조화

- EmotionType, CharacterId, ReactionCommand, StageCommand, ReactionData 같은 기본 모델을 만든다.
- 감정별로 애니메이션, 표정, 대사, 카메라, 조명, 사운드를 한 묶음으로 관리한다.
- 코드에서 if-else 로 모든 분기를 처리하지 말고, 명령-데이터-실행 흐름으로 정리한다.

5-4. 공간 연출 연동

- 캐릭터 반응과 함께 카메라 각도, 조명 색, 배경 사운드가 변하도록 연결한다.
- 캐릭터만 움직이는 데모에서 끝나지 않게 “전시 씬 전체가 반응한다”는 인상을 만든다.
- 이 단계부터 덱스터류 몰입형 콘텐츠 포트폴리오로 읽히는 힘이 생긴다.

5-5. 휴대폰 웹 패널 연결

- PC 에서 간단한 웹 서버를 띄우고, 휴대폰 브라우저가 동일 네트워크로 접속하도록 구성한다.

- 버튼 입력(인사/감정/분위기 전환)부터 붙이고, 텍스트 입력은 그 다음에 추가한다.
- 초기 통신은 REST API 또는 WebSocket 으로 시작한다. WebRTC·gRPC 는 지금 단계의 필수 구현이 아니라 추후 PoC 항목으로 둔다.

5-6. 실시간 명령 처리 레이어 정리

- 웹 패널 명령을 바로 Blueprint 에 꽂지 말고, 수신 → 해석 → 실행 흐름을 중간 레이어로 분리한다.
- 중복 입력 방지, 쿨다운, 현재 실행 중인 반응 우선순위를 정리하는 로직을 넣는다.
- 이 구조가 있어야 나중에 2 캐릭터, 복수 입력, 운영 모드, 시나리오 재생으로 확장된다.

5-7. 운영 모드/시연 모드 분리

- 운영 모드에서는 모든 명령 버튼을 노출하고, 시연 모드에서는 관람객용 최소 버튼만 노출한다.
- 테스트 중에는 현재 CharacterId, 현재 상태, 마지막 명령, 지연 시간 등을 화면에 표시하는 디버그 HUD 를 둔다.
- 이 항목은 포트폴리오에서 “실제로 굴릴 생각을 했다”는 인상을 준다.

5-8. 데모 시나리오 제작

- 시작/중간/하이라이트/종료 흐름이 있는 1~2 분짜리 시연 시퀀스를 만든다.
- 단순 기술 시연이 아니라, 관람객이 어떤 입력을 했고 화면이 왜 바뀌는지 이해되는 흐름으로 구성한다.
- 마지막에는 2 캐릭터 확장 계획(예: Character_02 가 같은 StageCommand 를 공유하면서 반응)을 문서로 제시한다.

5-9. 문서화와 패키징

- 기능 목록, 구조도, 상태 전이도, 향후 확장 항목, 시연 영상을 함께 정리한다.
- 포트폴리오 설명에서는 “1 캐릭터 MVP 이지만 2 캐릭터 확장 가능한 명령 구조와 상태 구조를 선행 설계했다”는 점을 명확히 쓴다.
- Bluegarage 방향성으로는 실시간 인터랙션 시스템, 덱스터 방향성으로는 몰입형 전시형 콘텐츠 시스템으로 각각 해석 가능한 표현을 준비한다.

6. 추천 아키텍처 요약

계층	책임	대표 기술
입력 계층	휴대폰 웹 패널, 명령 전송, 운영 모드/관람객 모드 분리	간단한 웹 UI, REST API 또는 WebSocket
도메인 계층	명령 해석, 상태 전이, 우선순위, 쿨다운, CharacterId·StageCommand 분리	C# 또는 별도 서비스, DTO/Resolver/Manager 구조

실행 계층	캐릭터 애니메이션, 표정, 대사 UI, 조명, 카메라, 사운드 실행	Unreal Blueprint, Animation Blueprint, Level Sequence
관측 계층	로그, 디버그 HUD, 시연 상태 확인	로그 출력, HUD 위젯, 테스트 패널

7. 지금 하지 않을 것 / 나중에 붙일 것

현재 비권장(초기 범위 제외)	후속 확장 후보
MetaHuman Animator 중심 고사양 파이프라인 실시간 얼굴 추적 WebRTC 미디어 스트리밍 실전 적용 2 캐릭터 전체 구조 동시 구현 복잡한 음성 인식/생성 자동화	2 캐릭터 상호작용 1 씬 WebSocket 기반 저지연 이벤트 HUD gRPC 또는 WebRTC PoC 운영 로그 저장 및 리플레이 장면 프리셋 전환 / 시나리오 편집기

8. 최종 정리

이 프로젝트의 본질은 “핸드폰으로 Unreal 화면을 조작하는 장난감”이 아니라, 입력-해석-캐릭터 반응-공간 연출이 이어지는 전시형 실시간 콘텐츠 시스템을 설계하는 데 있다.

초기 결과물은 1 캐릭터로 제한하지만, 구조는 CharacterId 중심으로 설계하여 향후 2 캐릭터·다중 오브젝트 시스템으로 확장 가능해야 한다.

포트폴리오 설명 시에는 Bluegarage 관점에서는 실시간 캐릭터 인터랙션 엔지니어링, 덱스터류 관점에서는 몰입형 전시형 콘텐츠 시스템 설계 역량으로 각각 번역해 제시한다.